BDD / Filosofía, Gherkin y Cucumber

```
Feature: Autenticación de usuario

  Scenario: Login exitoso
    Given el usuario está en /login
    When ingresa credenciales válidas
    Then accede al dashboard
```

✓ PASSED

Taller Semana 4 · QA Moderno

Jostin Alvarado · Jorge Ruiz

NotebookLM

Agenda

01 · **¿Qué es BDD? —**
El problema que resuelve

02 · **Los 3 Amigos —**
Negocio · Dev · QA

03 · **Lenguaje Gherkin —**
Sintaxis y palabras clave

04 · **Buenas Prácticas —**
Declarativo vs Imperativo

05 · **Antipatrones —**
Qué evitar al escribir

06 · **Cucumber & Serenity —**
Demo en vivo

sofka_

NotebookLM

Requisito

Negocio

El usuario debe poder iniciar sesión de forma segura

sofka_

NotebookLM

Requisito

Negocio

El usuario debe poder iniciar sesión de forma segura

Dev

????

sofka_

NotebookLM

El problema real

Tres equipos, tres interpretaciones distintas del mismo requisito

Negocio

El usuario debe poder iniciar sesión de forma segura

Dev

Implemento JWT con refresh token y OAuth2

QA

Pruebo el campo username y el botón Submit

💡 **Mismo requisito · tres interpretaciones · un software que no satisface a nadie.**

sofka_

NotebookLM

¿Qué es BDD?

Behaviour-Driven Development — una metodología, no solo un framework

TDD
Solo el dev escribe tests antes del código

→

ATDD
Tests de aceptación con el cliente

→

BDD
Lenguaje compartido entre todos

🗣️ **Colaboración entre roles**

Lenguaje natural estructurado

🔄 **Documentación viva y ejecutable**

sofka_

NotebookLM

Los 3 Amigos

Tres perspectivas obligatorias · un solo escenario acordado

Negocio / PO	Desarrollador	QA / Tester
¿Qué valor genera esto?	**¿Qué restricciones técnicas hay?**	**¿Qué puede salir mal?**
Define criterios de aceptación. Describe el escenario en lenguaje de negocio.	Identifica ambigüedades. Propone la solución técnica y conecta steps con código.	Propone casos de borde. Valida expectativas y automatiza con Cucumber.

sofka_

NotebookLM

Palabras clave Gherkin

Sintaxis legible por humanos · ejecutable por máquinas

Feature	Agrupa escenarios de la misma funcionalidad
Scenario	Un caso de uso concreto e independiente
Given	Precondición — estado inicial del mundo
When	Acción principal del usuario o sistema
Then	Resultado observable y verificable
And / But	Extiende el paso anterior
Background	Pasos comunes a todos los scenarios
Scenario Outline	Escenario parametrizado con tabla Examples

```
Feature: Carrito de compras
  Background:
    Given un producto "Laptop" a $80.000

  Scenario: Agregar producto disponible
    When agrego 2 unidades al carrito
    Then el carrito tiene 2 Laptops

  Scenario Outline: Total con descuento
    When agrego <cantidad> unidades
    Then el total es <total>
    Examples:
      | cantidad | total    |
      | 1        | $80.000  |
      | 2        | $144.000 |
```

sofka_

NotebookLM

Buenas Prácticas: Imperativo vs. Declarativo

Describe el QUÉ, no el CÓMO.

❌ Imperativo

```
abre el browser...
navega a la URL...
hace click en el campo...
escribe texto...
hace click en el botón Submit...
```

Describe clics y elementos del DOM.
Falla ante cualquier cambio menor de UI.
El Product Owner no puede validarlo.

Declarativo ✅

```
Given el usuario está en el login
When ingresa credenciales válidas
Then accede al dashboard
```

Describe la intención.
Sobrevive a cambios de interfaz.
El negocio lo lee y lo firma.

💡 **Regla práctica:** Si en tu escenario aparece una URL, un ID o un botón, estás siendo imperativo.

sofka_

NotebookLM

¿Es este ejemplo lenguaje Gherkin?

Gherkin

```
Feature: Gestión de promociones en el checkout

  Scenario: Aplicación de descuento por cupón promocional
    Given que tengo una "Sudadera de Algodón" en mi carrito
      And el precio total actual de la compra es "50.00€"
     When introduzco el código "PROMO10" en el campo de cupones
      And presiono la tecla "Enter" para confirmar
     Then el sistema debe actualizar el total a "45.00€"
      And se debe mostrar un mensaje de éxito en color verde
```


NotebookLM

Versión correcta - declarativa

Gherkin

```
Feature: Gestión de promociones en el checkout

  Scenario: Aplicación exitosa de un cupón de descuento porcentual
    Given que tengo un pedido elegible para promociones
     When aplico un cupón de descuento válido del 10%
     Then el precio total debe reflejar la rebaja correspondiente
      And el sistema debe confirmar que el cupón ha sido aceptado
```


NotebookLM

6 Antipatrones de Gherkin

Reconócelos · evítalos · corrígelos

Escenario Omnibus

Cubre múltiples comportamientos no relacionados.

Fix: 1 escenario = 1 comportamiento.

Steps Imperativos

Describe clics y DOM en lugar de intenciones.

Fix: Lenguaje de negocio, no de UI.

Exceso de And

Más de 3 And encadenados: el escenario hace demasiado.

Fix: Divide en varios Scenarios.

Step Conjuntivo

Given X and Y en un mismo paso viola la atomicidad.

Fix: Un step = una sola condición.

Datos en el Título

Parámetros concretos en el nombre del escenario.

Fix: Usa Scenario Outline + Examples.

Sin Given

Escenario sin contexto inicial; estado del sistema desconocido.

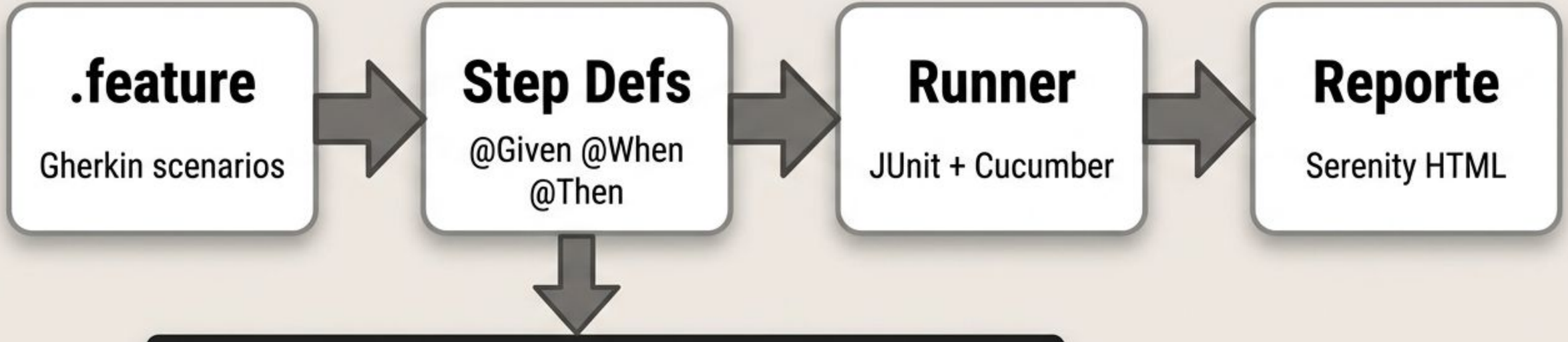
Fix: Siempre define la precondición.

sofka_

NotebookLM

El Puente Técnico: Cucumber & Serenity

Cómo el framework conecta lenguaje natural con código ejecutable




```
@Given("un producto {string} a ${double}")
public void crearProducto(String nombre, double precio) {
    producto = new Producto(nombre, precio, 10);
}
```

sofka_

NotebookLM

¡Gracias!

Better
happens.
Every day.

Key Takeaways

1. BDD es un proceso de comunicación, no solo una herramienta de testing.
2. Los 3 Amigos eliminan ambigüedades antes de escribir código.
3. Gherkin = lenguaje natural estructurado (Given / When / Then).
4. Declarativo: describe QUÉ hace el sistema, no CÓMO lo hace.
5. Cucumber conecta los .feature con código Java ejecutable.
6. Serenity genera documentación viva en cada ejecución.